

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

COURSE CODE: CSA1407

COURSE NAME: Compiler Design for Higher  
Level Processing

---

### COURSE OUTCOME COVERED

***CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)***

*Assessment Tool Weightage: 40%*

---

**QUESTIONS:4Total Marks:50**

Annexure A

### DESIGN TASK

---

***Code of Conduct:***

*I V.Aravind/192321119(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

*V.Aravind*

***Course Faculty***

***N.Deepa***

## Annexure A

### DESIGN TASK

#### Q1. Predictive Parser Design (Application Level)

**Task:** Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

##### Grammar:

$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \epsilon \quad T \rightarrow F T' \quad T' \rightarrow * F T' \mid \epsilon \quad F \rightarrow (E) \mid id$

##### Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

##### Test Case: Input String:

id + id \* id

##### Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted**

#### Q2. Shift-Reduce Parsing (Application Level)

**Task:** Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

##### Grammar:

$E \rightarrow E + E \quad E \rightarrow E * E \quad E \rightarrow id$

##### Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (\* has higher precedence than +)

**Test Case:Input:**

id + id \* id

**Expected Output:**

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

***Q3. Construct the LR parsing tables for the given grammar and perform a detailed comparison of different LR parsing techniques (SLR, LALR, and CLR), highlighting their construction steps, state transitions, table sizes, and any conflicts encountered.***

$S \rightarrow L = R \mid R$

$L \rightarrow * R \mid id$

$R \rightarrow L$

**Tasks****(i) CLR (Canonical LR) Design**

Construct the canonical collection of LR(1) items

Build the CLR parsing table

Show all item sets and transitions

**(ii) SLR Parser Design**

Compute FIRST and FOLLOW sets

Construct the SLR parsing table

Identify and explain any shift-reduce or reduce-reduce conflicts

### (iii) LALR Parser Design

Merge compatible LR(1) states

Construct the LALR parsing table

Compare with CLR and show how states are reduced

### (iv) Parsing Demonstration

Using any one of the constructed parsing tables,

parse the input string:

id = \* id

Show:

Stack

Input buffer

Action (Shift / Reduce / Accept)

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
  - Grammar expressiveness
  - Ease of implementation
  - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

### Q 5. Advanced LL(1) Parser Evaluation and Design

**Task:** *Analyze and redesign the given grammar to make it suitable for LL(1) parsing, construct the predictive parsing table, and evaluate its correctness by parsing the input string.*

**Grammar:**  $S \rightarrow S + T \mid TT \rightarrow T * F \mid FF \rightarrow (S) \mid id$

**Conditions:**

- Eliminate left recursion completely
- Apply left factoring where necessary
- Compute FIRST and FOLLOW sets accurately
- Construct the LL(1) parsing table
- Justify whether the grammar satisfies LL(1) conditions
- Trace full parsing steps with stack, input, and action

**Test Case:** Input:  $id + id * id$  Expected: String Accepted with full trace

## Q6. Comparative Analysis of SLR vs LALR vs CLR

**Task:** *Construct and compare SLR, LALR, and CLR parsing tables for the given grammar and evaluate their differences in handling conflicts and efficiency.*

**Grammar:**  $S \rightarrow L = R \mid RL \rightarrow * R \mid idR \rightarrow L$

**Conditions:**

- Construct LR(0), LR(1) item sets
- Compute FOLLOW sets (for SLR)
- Build SLR, LALR, and CLR tables
- Identify and explain conflicts in each method
- Compare number of states and table sizes
- Justify which method is most efficient

**Test Case:** Input:  $id = * id$  Expected: Valid parse with comparison analysis

## Q7. Operator Precedence Parsing Design and Evaluation

**Task:** *Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.*

**Grammar:**  $E \rightarrow E + E \mid E * E \mid (E) \mid id$

**Conditions:**

- Define operator precedence relations ( $<$ ,  $=$ ,  $>$ )
- Construct precedence table
- Handle associativity rules explicitly
- Detect and resolve conflicts

- Show step-by-step parsing actions
- Evaluate correctness of parsing decisions

**TestCase:** Input: id + id \* id

## Q8. Ambiguity Removal and Parsing Strategy Evaluation

**Task:** *Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.*

**Grammar:**  $E \rightarrow E - E \mid E / E \mid \text{id}$

**Conditions:**

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

**TestCase:** Input: id - id / id

## Q9. PEG vs CFG Parsing Strategy Evaluation

**Task:** *Design a PEG-based parser for the given expression grammar and critically evaluate its behavior compared to CFG-based parsing.*

**Grammar (CFG form):**  $E \rightarrow E + T \mid TT \rightarrow T * F \mid FF \rightarrow \text{id}$

**Conditions:**

- Convert CFG into PEG form
- Explain ordered choice behavior
- Demonstrate parsing steps
- Compare ambiguity handling with CFG
- Evaluate performance and backtracking
- Justify advantages and limitations

**Test Case:** Input: id + id \* id

## Q10. Design a Hybrid Parsing Strategy

**Task:** *Design a hybrid parsing system that combines LL(1) and LR parsing techniques to efficiently parse a programming language with both simple and complex constructs.*

**Conditions:**

- Identify which parts use LL(1) vs LR parsing
- Justify the hybrid design choice
- Define grammar transformations required
- Illustrate parsing workflow with diagram
- Demonstrate parsing of a sample input
- Evaluate advantages over standalone methods

**TestCase:** Input:  $\text{id} + \text{id} * \text{id}$

### Q11. Create an Unambiguous Grammar with Custom Operators

**Task:** Design an unambiguous CFG for a language supporting custom operators ( $\%$ ,  $\wedge$ ,  $+$ ,  $-$ ) with user-defined precedence and associativity, and construct a suitable parser.

#### Conditions:

- Define precedence hierarchy
- Handle both left and right associativity
- Eliminate ambiguity completely
- Construct parsing table (LL or LR)
- Validate grammar correctness
- Demonstrate parsing with detailed steps

**TestCase:** Input:  $\text{id} + \text{id} \% \text{id} \wedge \text{id}$

### Q12. Develop a New Parsing Algorithm

**Task:** Propose and design a new parsing algorithm that improves efficiency or readability compared to existing LL and LR techniques.

#### Conditions:

- Define algorithm steps clearly
- Compare with LL and LR methods
- Analyze time and space complexity
- Demonstrate parsing on sample grammar
- Highlight improvements and trade-offs
- Provide pseudo-code implementation

**TestCase:** Input:  $\text{id} * \text{id} + \text{id}$

### Q13. Design a Compiler Front-End for Expression Language

**Task:** Create a complete front-end design for a simple expression language including lexical analyzer, parser, and intermediate code generator.



**Conditions:**

- Define token specifications
- Design grammar and parser (LL/LR)
- Generate syntax tree
- Produce three-address code
- Handle syntax errors
- Demonstrate full compilation process

**Test Case:**Input:  $a = b + c * d$

**Q14. Create a PEG-Based Parsing System**

**Task:***Design a parsing system using Parsing Expression Grammars (PEG) for a programming language and evaluate its performance against CFG-based parsers.*

**Conditions:**

- Convert CFG to PEG format
- Handle ordered choice and backtracking
- Implement parsing logic
- Compare ambiguity handling
- Analyze performance and limitations
- Demonstrate parsing process

**Test Case:**Input:  $id + id * id$

## RUBRICS FOR CALCULATION:

| Criteria                      | Weightage | Level 4 – Exemplary                                                              | Level 3 – Proficient                                        | Level 2 –<br>Developing                      | Level 1 –<br>Beginning                 |
|-------------------------------|-----------|----------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------|----------------------------------------|
| Understanding of Concepts     | 20        | Demonstrates thorough understanding and integrates multiple concepts effectively | Good understanding with proper integration of most concepts | Basic understanding with partial integration | Poor understanding with no integration |
| Design & Implementation       | 25        | Well-designed solution with systematic and optimal implementation                | Correct design with minor issues                            | Basic design with errors or inefficiencies   | Incorrect or incomplete design         |
| Parser/Algorithm Construction | 20        | Accurate construction of parser/algorithm with complete logic                    | Mostly correct construction with minor errors               | Partial construction with missing logic      | Incorrect or incomplete construction   |
| Analysis & Validation         | 15        | Insightful analysis with correct validation and justification                    | Good analysis with reasonable justification                 | Basic analysis with limited validation       | Poor or incorrect analysis             |
| Documentation & Presentation  | 20        | Well-structured documentation with diagrams, steps, and clarity                  | Organized documentation with required details               | Basic documentation with missing details     | Poor or incomplete documentation       |

# Q1. Predictive Parser Design (Application Level)

## 1. Introduction & Left Recursion Check

Predictive parsing is a top-down parsing technique that uses a parsing table without backtracking. The given grammar is already free of left recursion and left factoring, making it immediately suitable for LL(1) table construction.

## 2. FIRST and FOLLOW Sets

- **FIRST(E)** = FIRST(T) = FIRST(F) = { ( , id }
- **FIRST(E')** = { +, ε }
- **FIRST(T')** = { \*, ε }
- **FOLLOW(E)** = FOLLOW(E') = { \$, ) }
- **FOLLOW(T)** = FOLLOW(T') = FIRST(E') \ {ε} ∪ FOLLOW(E) = { +, \$, ) }
- **FOLLOW(F)** = FIRST(T') \ {ε} FOLLOW(T) = { \*, +, \$, ) }

## 3. Step-by-Step Parsing Trace for $id + id * id \$$

| Stack        | Remaining Input | Action / Production Applied |
|--------------|-----------------|-----------------------------|
| \$ E         | id + id * id \$ | \$E \rightarrow T E'\$      |
| \$ E' T      | id + id * id \$ | \$T \rightarrow F T'\$      |
| \$ E' T' F   | id + id * id \$ | \$F \rightarrow id\$        |
| \$ E' T' id  | id + id * id \$ | Match id                    |
| \$ E' T'     | + id * id \$    | \$T' \rightarrow \epsilon\$ |
| \$ E'        | + id * id \$    | \$E' \rightarrow + T E'\$   |
| \$ E' T +    | + id * id \$    | Match +                     |
| \$ E' T      | id * id \$      | \$T \rightarrow F T'\$      |
| \$ E' T' F   | id * id \$      | \$F \rightarrow id\$        |
| \$ E' T' id  | id * id \$      | Match id                    |
| \$ E' T'     | * id \$         | \$T' \rightarrow * F T'\$   |
| \$ E' T' F * | * id \$         | Match *                     |

| Stack       | Remaining Input | Action / Production Applied |
|-------------|-----------------|-----------------------------|
| \$ E' T' F  | id \$           | \$F \rightarrow id\$        |
| \$ E' T' id | id \$           | Match id                    |
| \$ E' T'    | \$              | \$T' \rightarrow \epsilon\$ |
| \$ E'       | \$              | \$E' \rightarrow \epsilon\$ |
| \$          | \$              | <b>Accept</b> [cite: 1]     |

**Conclusion:** The string has been successfully parsed using the LL(1) parsing table without any conflict. Final result: **String Accepted**[cite: 1].

# Q2. Shift-Reduce Parsing (Application Level)

**Grammar:**  $E \rightarrow E + E \mid E * E \mid id$  [cite: 1]

**Conditions:** Resolve conflicts using operator precedence where  $*$  has higher precedence than  $+$  [cite: 1].

**Test Case Input:**  $id + id * id$  [cite: 1]

## 1. Introduction

Shift-reduce parsing is a bottom-up technique that attempts to construct a parse tree from the leaves (input) up to the root (start symbol) using two primary operations: **Shift** and **Reduce**.

## 2. Complete Parsing Trace

| Stack         | Input Buffer    | Action                         |
|---------------|-----------------|--------------------------------|
| \$            | id + id * id \$ | Shift                          |
| \$ id         | + id * id \$    | Reduce $E \rightarrow id$      |
| \$ E          | + id * id \$    | Shift                          |
| \$ E +        | id * id \$      | Shift                          |
| \$ E + id     | * id \$         | Reduce $E \rightarrow id$      |
| \$ E + E      | * id \$         | Shift (Precedence: $*$ > $+$ ) |
| \$ E + E *    | id \$           | Shift                          |
| \$ E + E * id | \$              | Reduce $E \rightarrow id$      |
| \$ E + E * E  | \$              | Reduce $E \rightarrow E * E$   |
| \$ E + E      | \$              | Reduce $E \rightarrow E + E$   |
| \$ E          | \$              | Accept [cite: 1]               |

## 3. Explanation of Precedence Conflict Resolution

When the stack contains  $E + E$  and the lookahead is  $*$ , a standard ambiguity conflict arises. Since multiplication has higher precedence than addition, the parser chooses to **Shift** the  $*$  operator rather than reducing  $E + E$  prematurely. This guarantees correct operator hierarchy and association.

**Conclusion:** The expression was accurately reduced following correct precedence rules. Final result: **String Accepted**[cite: 1].

# Q4. Critical Evaluation: Parsing Expression Grammars (PEG)

**Task:** Critically evaluate PEG as a modern parsing technique compared to traditional LL and LR parsers[cite: 1].

## 1. Ambiguity Handling & Ordered Choice

Unlike Context-Free Grammars (CFGs) which are inherently ambiguous in many natural structures, PEGs use **ordered choice** (denoted by `/` instead of `|`). The first matching alternative is unconditionally chosen, and failing alternatives are never backtracked past unless required. This completely eliminates syntactic ambiguity by design[cite: 1].

## 2. Comparative Analysis Matrix

| Feature                    | Traditional LL(k) / LR(k)                                                        | Parsing Expression Grammar (PEG)                                                              |
|----------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Grammar Expressiveness     | Limited to subset of CFGs (requires strict factoring/recursion removal).         | Can express all DCFLs and many non-context-free languages (e.g., matching nested constructs). |
| Implementation Complexity  | High table generation complexity; requires external lexer generators (Lex/Yacc). | Simpler recursive descent implementation; lexing and parsing are unified.                     |
| Performance & Backtracking | Deterministic linear time ( $O(n)$ ) without backtracking.                       | Can experience exponential time unless optimized with memoization (Packrat parsing).          |

## 3. Real-World Adoption & Limitations

**Python's PEG Parser:** Starting in Python 3.9, the CPython implementation transitioned from an LL(1) parser to a PEG parser. Python's developers preferred PEG because it removes the artificial separation between the lexer and parser, handles tricky syntactic lookaheads seamlessly, and allows flexible grammar evolution without grammar restriction constraints[cite: 1].

**Limitations:** PEGs struggle with un-memoized left recursion (though modern variations support it) and lack native semantic feedback loops found in LR engines[cite: 1].

---

**Conclusion:** PEG is highly suitable for modern programming languages requiring expressive and clean syntax specification, though memory overhead via memoization must be managed.

---



## Q10. Design of a Hybrid Parsing Strategy

**Task:** Design a hybrid parsing system combining LL(1) and LR parsing techniques for simple and complex constructs[cite: 1].

**Test Case Input:** `id + id * id`[cite: 1]

### 1. Hybrid Architecture Justification

Standard compilers usually commit to either a top-down or bottom-up approach. However, real-world programming languages contain distinct sections: lightweight headers or simple declarations (best parsed efficiently via **LL(1) predictive parsing**) and deeply nested, operator-heavy expressions (best handled robustly via **LR bottom-up parsing**). A hybrid router optimizes throughput by routing token streams dynamically.

### 2. Workflow & Design Components

- **Lexical Analyzer:** Scans the source file and emits a continuous stream of tokens.
- **Dispatcher Router:** Inspects syntactic context tags or statement scopes. Simple declarations are directed to the LL(1) engine, while complex expressions are routed to the LR engine.
- **Unified Abstract Syntax Tree (AST):** Both sub-parsers merge their outputs into a uniform syntax representation structure.

### 3. Test Case Processing: `id + id * id`

When processing `id + id * id`, the dispatcher identifies the expression as operator-intensive and routes it to the **LR Parser core**. The LR parser applies shift- reduce actions, handles operator precedence correctly (\* over +), and yields a valid AST node structure.

**Conclusion:** The hybrid model maximizes performance speed and avoids the parsing conflicts typically encountered when forcing an entire complex language grammar into a single rigid parser architecture.

# Infographic Concept Map: Hybrid Parsing Architecture (Q10)

